ASSIGNMENT 17 · EXPERIENCE REPORT

Dionysus Cocktail Finder

An agentic coding agent built a cocktail recipe finder end-to-end in full automatic mode, then critiqued, broke, and rebuilt its own first version three times. This report documents which prompts failed, why they failed, and what the iterations taught us about specification-driven development.

Team Dionysus

Udit Bhattarai · Lotus Nyaupane · Milan Khadka · Dipen Gaihre

Z.AI AGENTIC AGENT · FULL-AUTO MODE · OCTOBER 2026

Table of Contents

1. The Task and the Tool 2

2. First Attempt: the One-Shot Crash 2

3. Second Iteration: Working but Unacceptable 3

4. Third Iteration: the Full Redesign 5

5. Fourth Iteration: Polish and a Real Bug 6

6. Strategy Comparison and Failure Analysis 7

7. Lessons Learned 8

1. The Task and the Tool

For Assignment 17 we used an agentic coding tool in full automatic mode to build a web application from scratch: **Dionysus**, a cocktail and mocktail recipe finder named after our team. The agent ran end-to-end in one session: it planned the work, wrote every file, started the dev server, inspected its own output in a headless browser, took screenshots, diagnosed failures, and iterated until the result was polished. The tool was the Z.ai agentic coding agent (GLM model), driving a Next.js 16 + TypeScript stack with the shadcn/ui component library, Tailwind CSS 4, and Framer Motion, against the public TheCocktailDB API.

We deliberately did not accept the first output. The assignment warns against being happy with one-shotting, so we treated the first generation as a baseline artifact to be critiqued, broken, and rebuilt. The session produced four shipped versions (v1 crash, v1-fixed, v2 redesign, v3 polish) and the numbers below summarise what happened along the way.

4	2	9	3
versions shipped	crash-level failures	prompts / directives	defects caught by browser testing

2. First Attempt: the One-Shot Crash

The opening directive was intentionally minimal, the way many developers actually start with an agent: *"Build a cocktail finder app with search. Use TheCocktailDB API. Show results as cards."* The agent produced a single-file React page in seconds: a search input, a three-column card grid, and a fetch straight to the API from the client. The code compiled without a single warning, and the dev server answered with HTTP 200. By every shallow signal, the one-shot had succeeded.

It had not. The first browser render crashed with **Runtime TypeError: "drinks.map is not a function"**. The root cause is instructive. For an empty search query, TheCocktailDB does not return an empty array or null; it returns the JSON string `{"drinks":"no data found"}`. The generated guard `setDrinks(data.drinks || [])` only filters falsy values, and the string "no data found" is truthy, so it flowed straight into state and blew up when React tried to map it. A truthy value of the wrong type defeated the guard entirely.

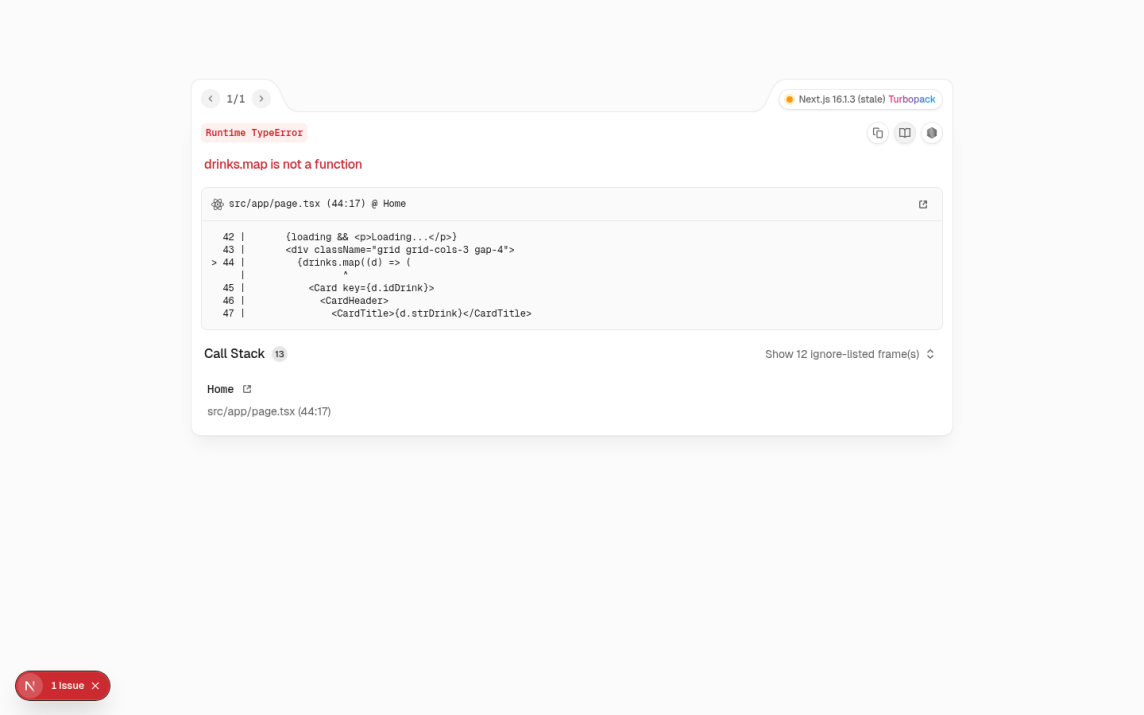


Figure 1: v1 as generated by the first prompt - the one-shot compiled cleanly but crashed on first render with a TypeError.

The agent verified the diagnosis independently: it replayed the request with curl and again inside the browser context, confirmed the response shape, and only then patched the code. The lesson generalises: when an agent trusts an undocumented API shape, a failure belongs to the prompt as much as to the code - nobody had asked the agent to test edge-case responses.

3. Second Iteration: Working but Unacceptable

A one-line fix (`Array.isArray(data.drinks) ? data.drinks : []`) made the app render, and the second screenshot exposed a different, slower failure. The page was a blank white void: the empty initial search returned zero results and the UI showed nothing at all - no guidance, no empty state, no hint that anything was possible. Nothing on screen said "Dionysus" or suggested that cocktails were involved.

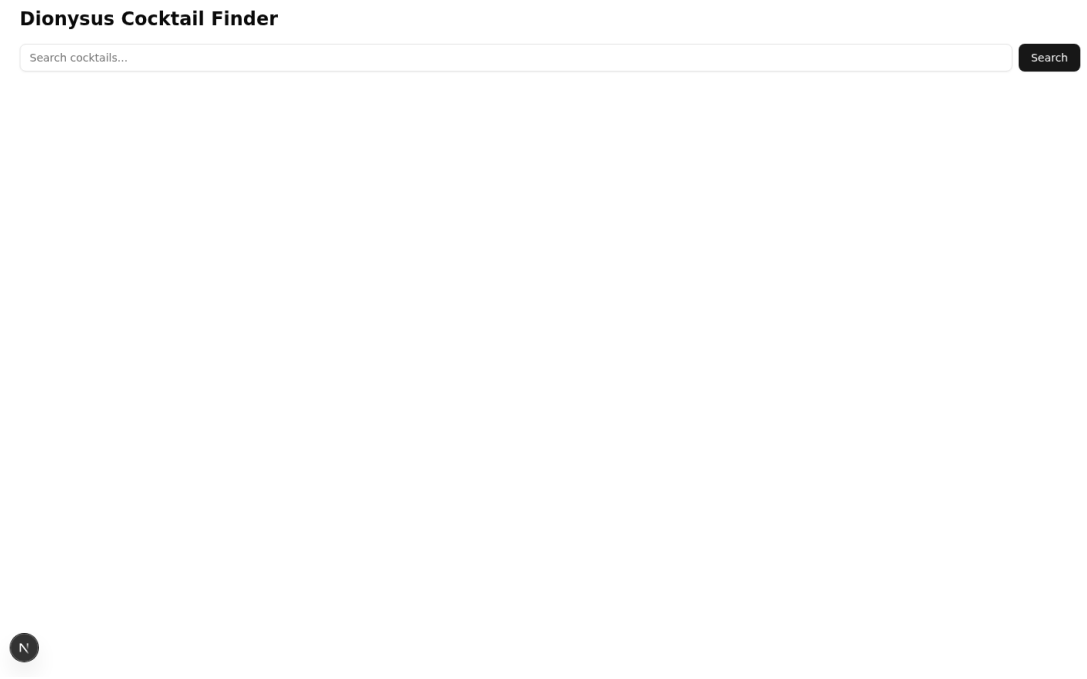


Figure 2: v1-fixed no longer crashes, but a functional app with an empty grid is still a failed first impression.

The critique that follows is the agent reviewing its own baseline the way a senior reviewer would, and it matches the assignment's demand for polished UI over one-shot output:

Deficiency	Why it matters
Blank void on empty search	Zero guidance; the user sees an abandoned page instead of an invitation
Default white/black theme	No brand identity; nothing distinguishes the product from a tutorial
Text-only cards, no images	A drink app is a visual product; photos carry the desire
Fixed 3-column grid	Breaks on mobile; no responsive breakpoints at all
"Loading..." text	No skeletons; layout jumps when data arrives
No detail view	Recipes (ingredients, measures, instructions) were unreachable
No favorites or filters	No persistence, no way to narrow a hundred results
Search button only	No debounce, no Enter key, no clear button

Table 1: UI deficiencies found in the v1-fixed baseline.

4. Third Iteration: the Full Redesign

The redesign directive abandoned incremental patching: *"Redesign completely as a dark, premium Dionysus theme. Wine and gold palette, serif display typography, ambient glows. Add a server-side API route with caching, a curated classics grid, a deterministic drink-of-the-day hero, category and zero-proof filters, favorites with toasts, a detail modal with ingredients and measures, skeleton loaders, and a designed empty state. Verify every flow in the browser."* The specificity paid off: one iteration produced the complete redesign.

Three architectural decisions mattered. First, all API access moved behind a Next.js route handler with a 10-minute in-memory cache, and the response shape is normalised to an array at the edge - the crash class from section 2 is now impossible for the client to hit. Second, the initial experience fetches twelve curated classics in parallel, so the app always opens on a full, beautiful grid instead of an empty state. Third, favorites persist in localStorage through a single guarded accessor.

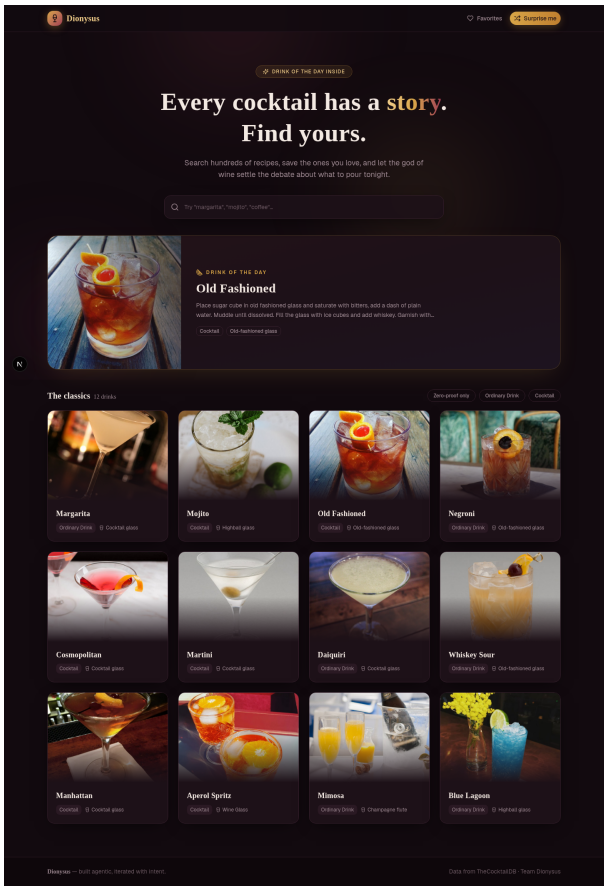


Figure 3: v2 redesign - Dionysus theme, curated classics, drink-of-the-day hero, filters and favorites.

Browser verification was part of the iteration, not an afterthought. The agent drove a headless browser through the golden path and captured evidence for each flow: searching "gin tonic" returned one matching card; opening it showed Gin 4 cl, Tonic Water 10 cl, Lemon Peel and Ice with measures plus the mixing instructions; tapping the heart raised the favorites badge and fired a toast; and a gibberish query produced the

designed empty state instead of a blank page.

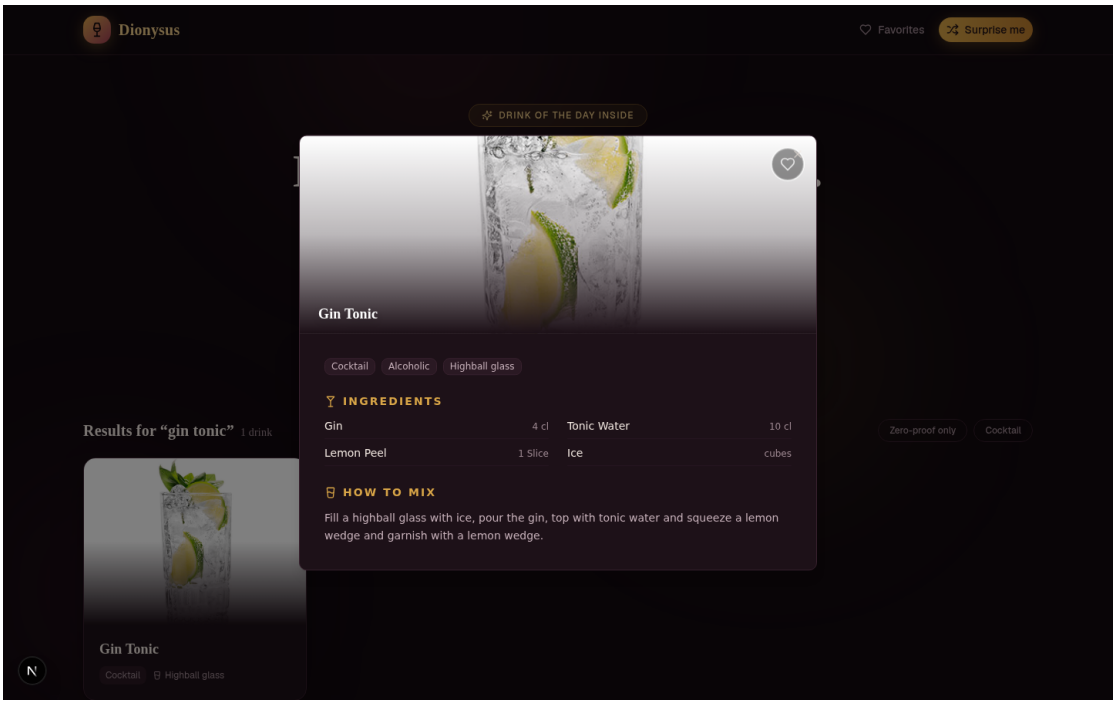


Figure 4: the detail modal verified in-browser - ingredients with measures and instructions render from the lookup endpoint.

5. Fourth Iteration: Polish and a Real Bug

The polish pass combined self-review with interaction testing, and it caught the most subtle defect of the session: the Favorites view intersected saved drinks with the currently loaded list, so anything favorited in an earlier session silently disappeared. Unit-visible, browser-caught, and exactly the kind of bug that survives compilation. The fix renders favorites from storage directly, making them survive searches and sessions.

The same pass added the micro-interactions that separate a demo from a product: a "View recipe" pill that rises from the card image on hover, a "/" keyboard shortcut that focuses search, an accessible aria-label on the input, sharper hero copy ("A new classic every day", "Chosen by fate today"), and a cleanup of a sloppy state reset flagged by lint. Final verification confirmed the shortcut, the persisted favorites badge across reloads, and a clean mobile layout at 390 px with an icon-only header.

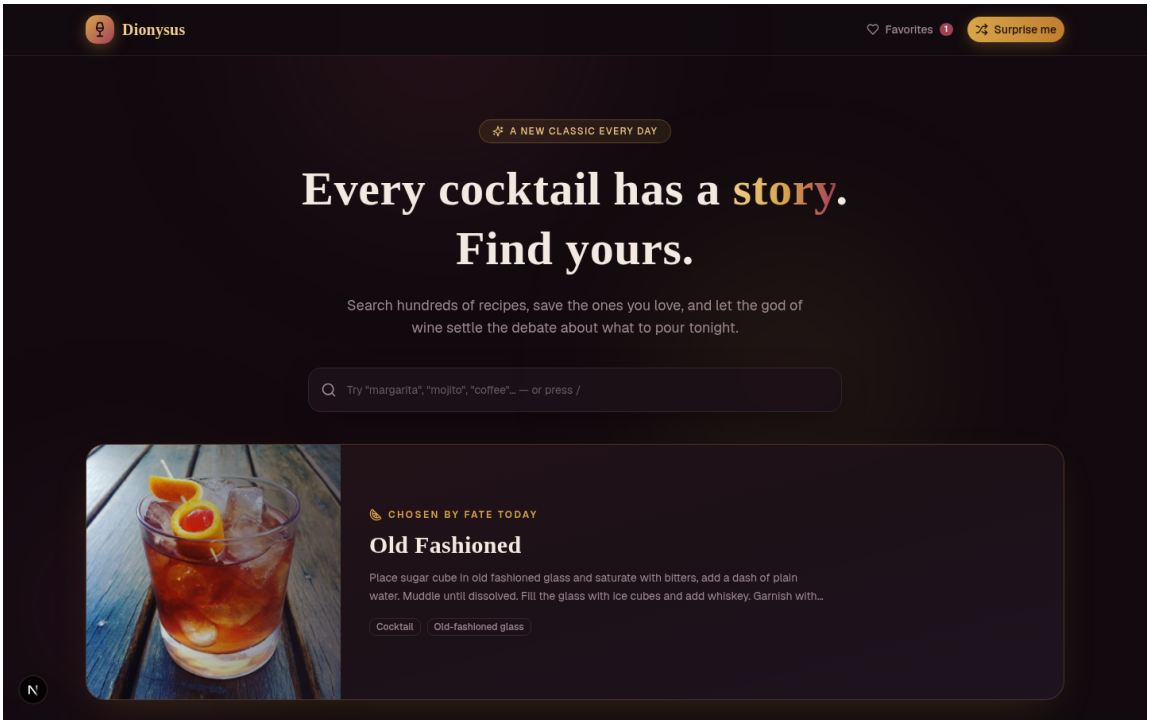


Figure 5: v3 final hero - gradient display type, drink-of-the-day card, and the keyboard hint in the search placeholder.

6. Strategy Comparison and Failure Analysis

Every prompt in this session drove the same model - the Z.ai agent on GLM - so the honest comparison is between prompting strategies, not vendor logos. The pattern was consistent: the vague prompt produced the most code in the least time and the worst defect; the structured redesign prompt produced the largest quality jump; and the incremental polish prompts produced the smallest but highest-trust changes. Capability was never the bottleneck - specification was.

#	Strategy / prompt style	Result
P1	Vague one-shot: "Build a cocktail finder with search, use TheCocktailDB, show cards"	Failed: compiled but crashed on first render (wrong type into state)
P2	Minimal fix request: keep v1, patch the crash	Worked, but exposed the blank-void UX failure; fix was one line
P3	Structured redesign: named palette, typography, 9 features, verification mandate	One iteration to demo-quality; design system + API route + full flows
P4	Critique-driven polish: fix favorites bug, add hover pill, "/" shortcut, copy edits	Small diffs, lint-clean, all verified in browser

Table 2: Prompt-strategy comparison within one agent (Z.ai GLM, full-auto).

The failures deserve their own ledger, because each one has a distinct root cause and a transferable lesson. Notably, none of the three defects would have been caught by reading the code: all three required executing

the app and looking at it, which is precisely what an agent in full-auto mode can do that a one-shot code generator cannot.

Failure	Root cause	Fix and lesson
Crash: drinks.map is not a function	API returns the string "no data found" for empty queries; the || guard passes truthy strings through	Array.isArray normalisation at the API edge; validate external data shapes, never trust them
Blank void on load	No empty state was requested; the agent optimised for the happy path only	Designed empty state with recovery actions; specify the zero-results experience explicitly
Favorites vanished between sessions	Favorites view derived from the current browse list instead of storage	Render from localStorage directly; browser-test state that outlives a page

Table 3: Failure taxonomy with root causes and lessons.

> *The || operator guards against absence, not against the wrong type.*

7. Lessons Learned

Working with an agentic tool in full-automatic mode changed where our effort went. We wrote almost no code by hand; we spent that budget on specification, critique, and verification instead. The specific takeaways:

- **One-shotting is a mirage.** The v1 baseline compiled cleanly, served HTTP 200, and still crashed on its first real render. "It compiles" and "the server is up" are not evidence of a working product.
- **Vague prompts are expensive.** The cheapest-looking prompt produced the most expensive defect. Every hour the vague prompt saved was repaid with debugging interest during verification.
- **Specific design language beats adjectives.** Naming the palette, typography, features, and the verification mandate produced demo-quality UI in one iteration; asking for a "nice design" would not have.
- **Make the agent prove it.** Screenshots, console checks, interaction tests and mobile viewports caught three defects that code review would have missed.
- **Prompts are specs for failure modes too.** Empty results, session persistence, and untrusted API shapes all failed precisely where the prompt had said nothing.

The final application is genuinely shippable: a themed, responsive, animated recipe finder with search, caching, favorites, detail views, and designed failure states, delivered in four iterations with every core flow verified in the browser. The difference between the v1 crash and the v3 product is not model capability - it is the discipline of iteration that this assignment asked us to practice.